

Problems

1) longer one will take a bit longer

2) x_1 - decode
 x_2 - prefill (new one)

input sz is different for both

We can group if OoP
Input shape are same

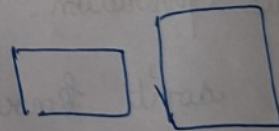
Soln 2: Split the Request

'selective batching'

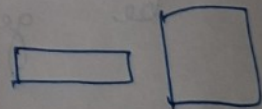
Piggy back Decoding

- decode maximal batching

Prefill - compute bound



Decode - memory bound

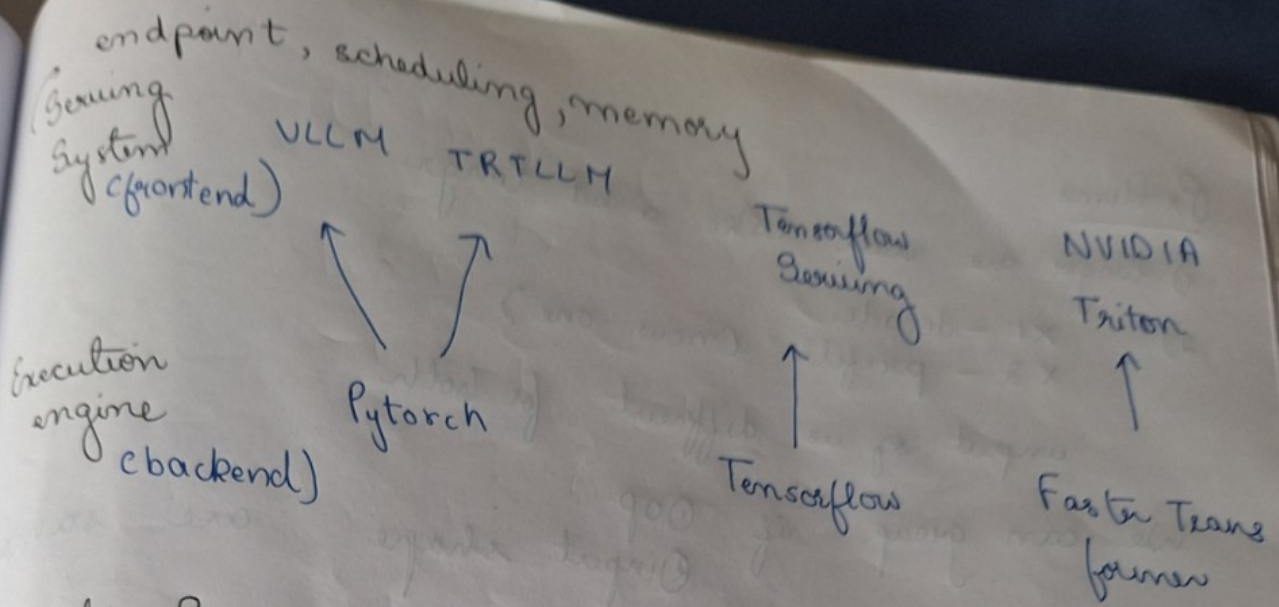


Batch
prefill +
decode

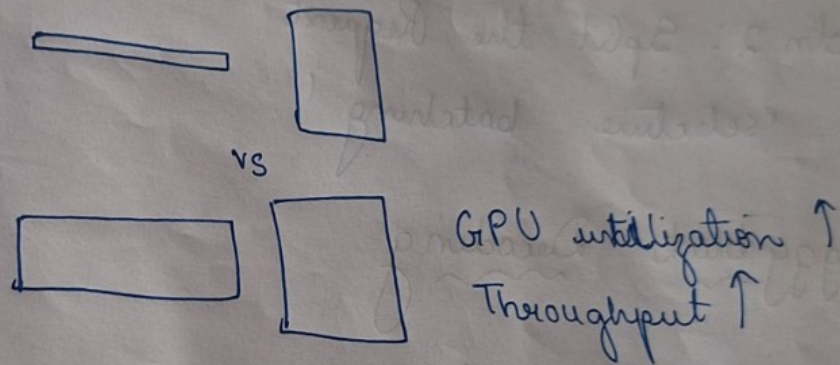
free side model weights

↑ Throughput

Loading weights from global
to on-chip memory



Batching?



Autoregressive Generation: Even if input ^{sz} if fixed, we don't know how many tokens will be generated.

Cannot have static batching!

x1	I	think	this	is	great
x2	I	love	you	-	-

Continuous Batching (ORCA 2022)

after every iteration check for $\langle \text{EOS} \rangle$
sprinkle in new ones

LLM serving: what to remember

Batching, prefill/decode, scheduling, KV cache, and the latency/throughput trade-off

1. Prefill vs decode

Prefill processes the prompt in parallel and is usually compute-heavy. Decode produces one new token per active sequence per iteration and is usually memory-bandwidth-heavy.

2. Why batching helps

A larger batch lets the GPU do more useful work per model-weight read, raising utilization and throughput - especially during decode.

3. Why static batching is bad for generation

Output length is unknown. Short requests finish early while long ones keep the batch alive, causing padding, idle slots, and head-of-line blocking.

4. Continuous batching / iteration-level scheduling

After each generation step, finished requests leave and waiting requests can enter. The scheduler advances resident sequences one iteration at a time, not request-by-request.

5. Requests stay mathematically independent

Batching only adds a batch dimension or packed representation. Each sequence has its own attention positions, masks, and KV-cache entries; equal token counts do NOT mean equal KV caches.

6. Different sequence lengths are normal

Serving engines handle ragged lengths using padding, packing, paged KV layouts, or kernels that accept per-sequence metadata. Requests do not need identical histories.

7. Long prefill can stall decode

A huge prompt can monopolize an iteration. Chunked prefill caps how many prompt tokens are processed at once so ongoing decodes keep emitting tokens.

8. Selective batching (Orca idea)

Batch operations when requests can share the same computation shape/semantics, and keep request-specific work separate. This avoids forcing every sequence to advance identically.

9. Latency vs throughput is the central trade-off

Bigger batches usually improve throughput, but excessive queueing or prefill work can hurt TTFT and inter-token responsiveness.

10. Know the metrics

TTFT = arrival to first token. TBT = gap between consecutive output tokens. TPOT = average time per output token. Tail TBT (for example p99) exposes stalls that a mean TPOT can hide.

11. KV cache is often the concurrency limit

KV memory grows roughly linearly with active context length and model size. More resident requests improve batching until KV/cache memory becomes the bottleneck.

12. Useful mental model

Serving system = request frontend + scheduler + model executor + KV/memory manager + optimized kernels. Systems such as vLLM or TensorRT-LLM combine these pieces differently.